Intermediate Flask + PyMongo Coding Challenge

Student Assignment Tracker API

Difficulty: Intermediate **Expected Time:** ~30 minutes

Scenario

You are building a backend for an online learning platform. The system needs to store student information and track the assignments they submit. Build a REST API using Flask and MongoDB (PyMongo) to manage this data.

Student Document Structure

```
{ "_id": "ObjectId", "name": "Rishabh", "email": "rishabh@example.com", "course": "Python",
"assignments": [ { "title": "Flask API", "score": 85 } ] }
```

Requirements

1. Add a Student — POST /students

Request body:

```
{ "name": "Rahul", "email": "rahul@example.com", "course": "Python" }
```

Requirements: name, email, and course are required. Email must be unique. Initially, assignments should be an empty list. Return an appropriate success response.

2. Get All Students — GET /students

Return all students stored in the database.

```
[ { "_id": "...", "name": "Rahul", "email": "rahul@example.com", "course": "Python",
"assignments": [] } ]
```

3. Add an Assignment — POST /students/<student_id>/assignments

Request body:

```
{ "title": "MongoDB Assignment", "score": 90 }
```

Requirements: the student must exist; title and score are required; score must be between 0 and 100; add the assignment to the student's assignments array. Use an appropriate MongoDB update operation such as $push.

4. Get Students Who Scored Above a Given Score — GET /students/top-performers/<int:score>

For example: GET /students/top-performers/80. Return students who have at least one assignment with a score greater than or equal to the given score.

Bonus ■

GET /students/<student_id>/average-score

Calculate and return the student's average assignment score.

```
{ "student": "Rahul", "average_score": 86.67 }
```

Technical Requirements

Requirement	Expected
Framework	Flask
Database	MongoDB
Driver	PyMongo
Request handling	request.json / request.get_json()
Database operations	Use MongoDB operations; do not store data in Python lists
Validation	Validate required fields and score range
Error handling	Handle invalid/non-existent student IDs
HTTP	Return appropriate HTTP status codes
Code quality	Keep the implementation reasonably modular

Skills Being Tested

Skill	Tested
Flask routes	Yes
GET / POST APIs	Yes
JSON request handling	Yes
PyMongo insert_one()	Yes
PyMongo find()	Yes
PyMongo update_one()	Yes
MongoDB $push	Yes
MongoDB array querying	Yes
ObjectId handling	Yes
Input validation	Yes
HTTP status codes	Yes
Aggregation / average calculation	Bonus

Suggested Marking Scheme

Component	Marks
Add Student API	2
Get All Students API	2
Add Assignment API	3
Top Performers API	2

Validation & Error Handling	1
Total	**10**